

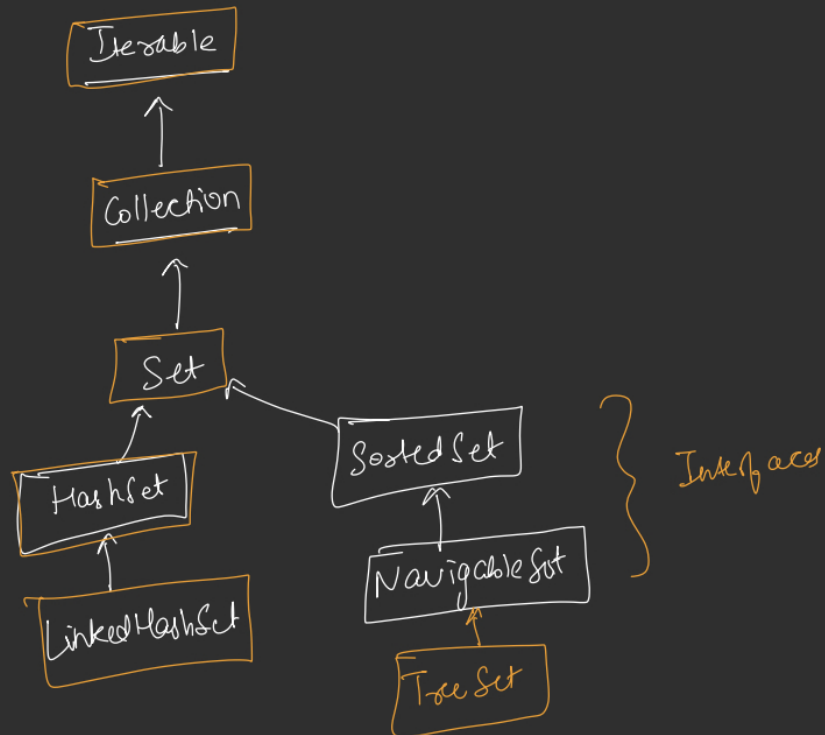
E first()
 E last()
 $\text{SortedSet} \langle E \rangle$ headSet (E el)
 $\text{SortedSet} \langle E \rangle$ tailSet (E el)
 $\text{SortedSet} \langle E \rangle$ subSet (E from, E to)

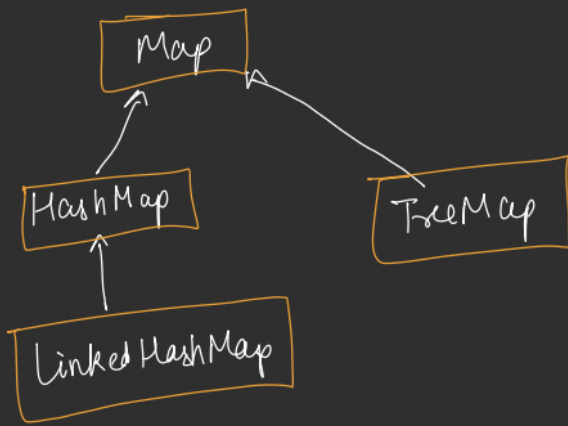
Sorted Set

E lower (E e)
 E floor (E e)
 E ceiling (E e)
 E higher (E e)
 E pollFirst()
 E pollLast()
 $\text{NavigableSet} \langle E \rangle$ descendingSet()
 $\text{Iterator} \langle E \rangle$ descendingIterator()
 $\text{NavigableSet} \langle E \rangle$ headSet (E el, boolean b1ap)
 $\text{NavigableSet} \langle E \rangle$ tailSet (E el, boolean b1ap)
 $\text{NavigableSet} \langle E \rangle$ subSet (E from, boolean fromInc, E to, boolean toInc).

Navigable Set

Set





HashTable

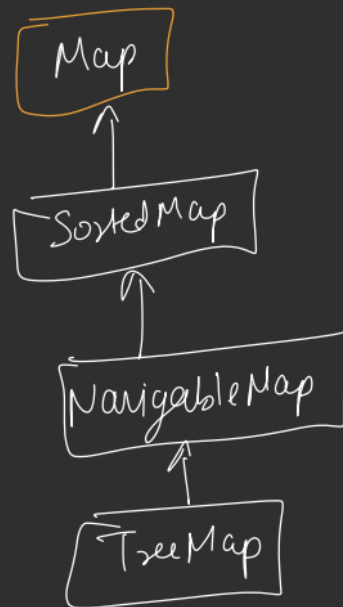
Properties

EnumMap

IdentityMap

ConcurrentHashMap

WeakHashMap



HashTable

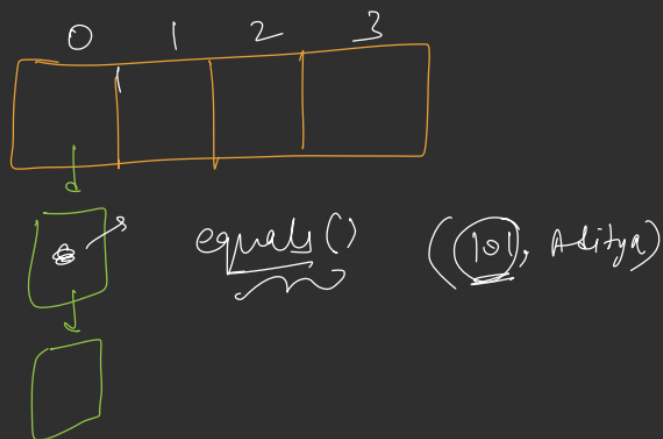
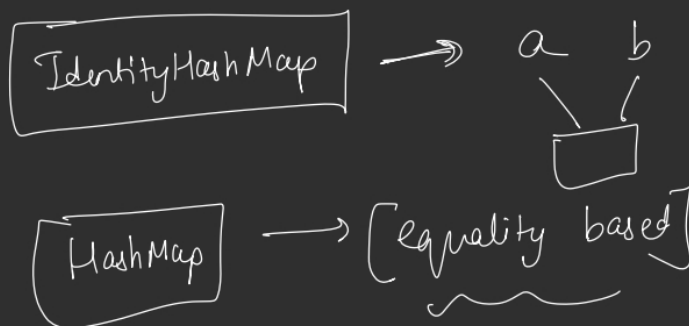
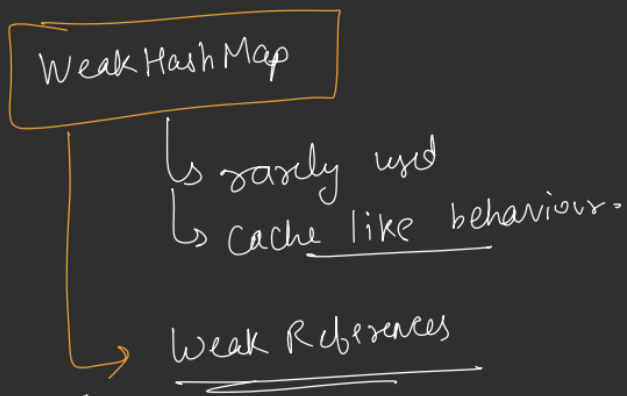
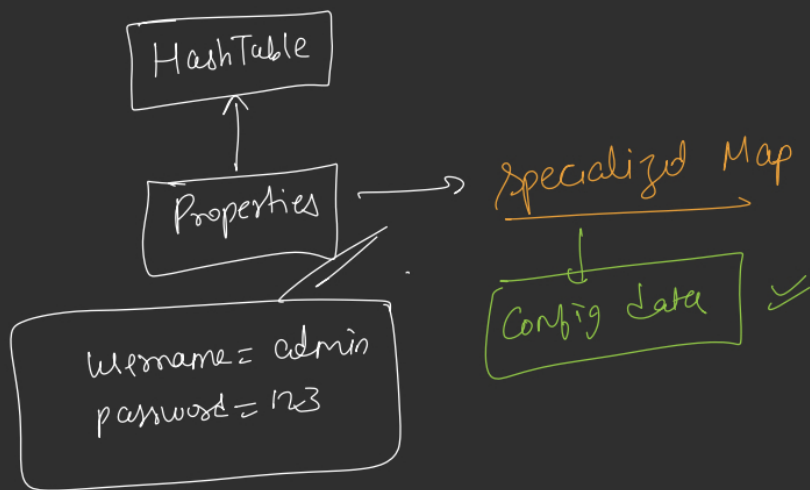
↳ legacy class

→ HashMap

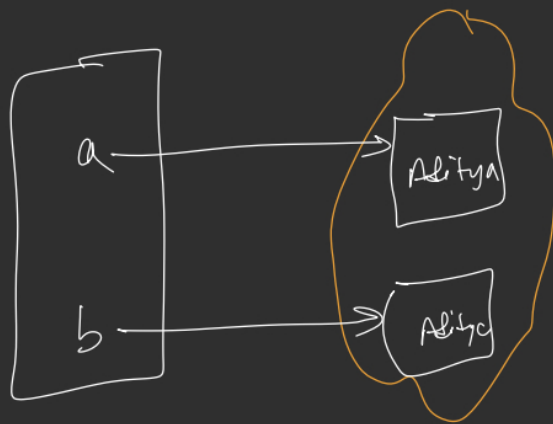
→ Thread safe

→ Overhead → slow

Vector



```
String a = new String("Aditya");
String b = new String("Aditya");
```



```
a.equals(b);
```

```
↓
True
```

```
a.hashCode() ==
b.hashCode()
```

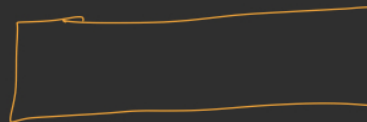
IdentityHashMap



a == b ✓

EnumMap

```
enum Day {
    MON,
    TUE,
    WED,
    ...
}
```



```
Map<Day, Integer> mp =
    new HashMap<>();
```

- Hashing tech (x)
- Memory efficient ✓
- null Key ~~are~~ not allowed (x)
- null Value (✓)

```
mp.put(MON, 1);
mp.put(TUE, 2);
```

```
Map<Day, Integer> map =
```

```
new EnumMap<>();
```

Iteration order is preserved. ✓✓

Concurrent Hash Map

↳ Thread-safe Map

HashTable